Retrieval-Augmented Generation (RAG)

From Concepts to Code: Building AI Systems That Know Your Data

Workshop • 3 Hours • Hands-On

Have you heard about GenAI?

Have you worked on building an application using LLM?

Workshop Agenda

5 modules • 3 hours

01

Foundations of RAG

LLM limits, what is RAG, embeddings, vector DBs, chunking

02

Build Your First RAG System

Hands-on: LangChain + OpenAI + ChromaDB end-to-end pipeline

—

Break

10-minute break

03

Advanced RAG Techniques

Re-ranking, multi-query, hybrid search, memory, evaluation

04

RAG in Production & the Future

Agentic RAG, Graph RAG, multimodal, modern production stack

05

Wrap-Up & Q&A

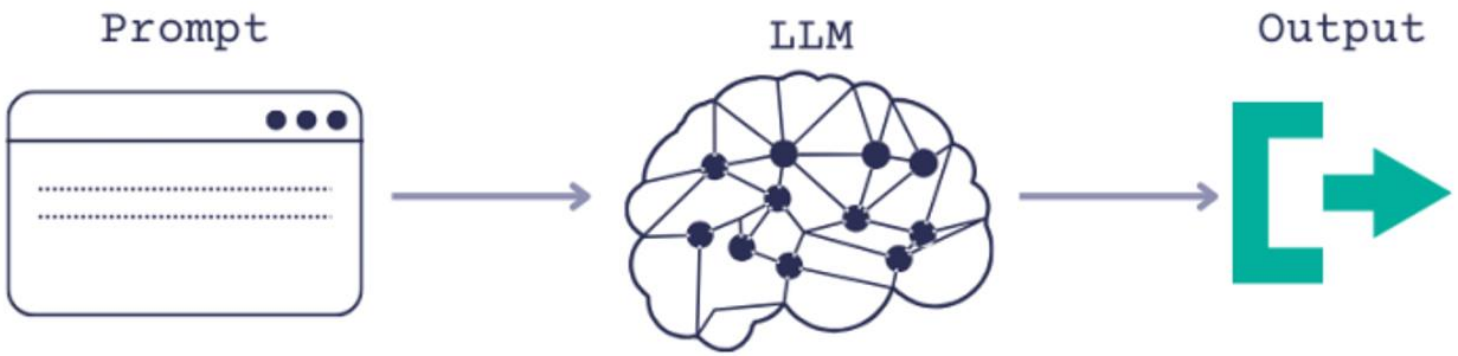
Key takeaways, resources, questions

Content Generation

Conversational AI

Writing Code

Conversational AI using LLM



The Problem

Module 1: Foundations

You ask an LLM about your company's internal docs...

"What is our refund policy for enterprise contracts?"

You

"Based on standard enterprise practices, your refund policy likely allows a 30-day window with approval from the account manager..."

LLM

HALLUCINATION

No Access to Private Data

LLMs are trained on public internet data. Your internal documents, databases, and APIs are invisible to them.

Knowledge Has a Cutoff

The model stops learning after training. Yesterday's product update, today's pricing - the LLM doesn't know.

Confidently Wrong

LLMs generate plausible-sounding answers even when they have no real knowledge. This is hallucination.

LLM Limitations

Why do base LLMs fall short in production?

01

Static Knowledge

Knowledge Cutoff

Training data has a fixed end date. The model has no awareness of events, documents, or updates that occurred after its training window closed.

02

Trust Risk

Hallucinations

LLMs predict the most probable next token. They don't verify facts. When they lack knowledge, they invent plausible-sounding (but wrong) answers.

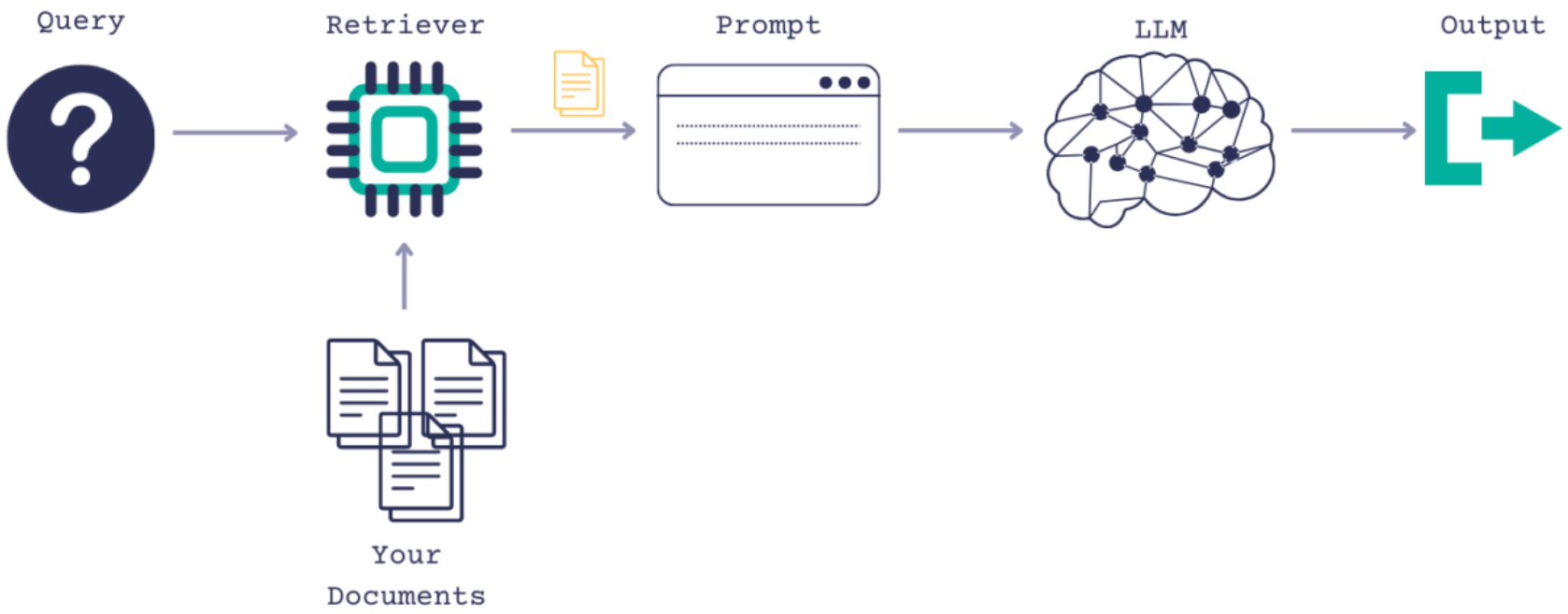
03

Data Blind

No Private Data Access

Your PDFs, databases, Slack messages, and internal wikis were never in training data. The LLM cannot reason over what it has never seen.

Conversational AI using RAG



What is RAG?

Retrieval-Augmented Generation

Give the LLM what it needs to know - at the time it needs to know it.

1

Retrieve

Fetch relevant context from your private documents using semantic search

2

Augment

Inject the retrieved context into the LLM prompt alongside the user's question

3

Generate

The LLM generates a grounded, factual answer backed by YOUR data

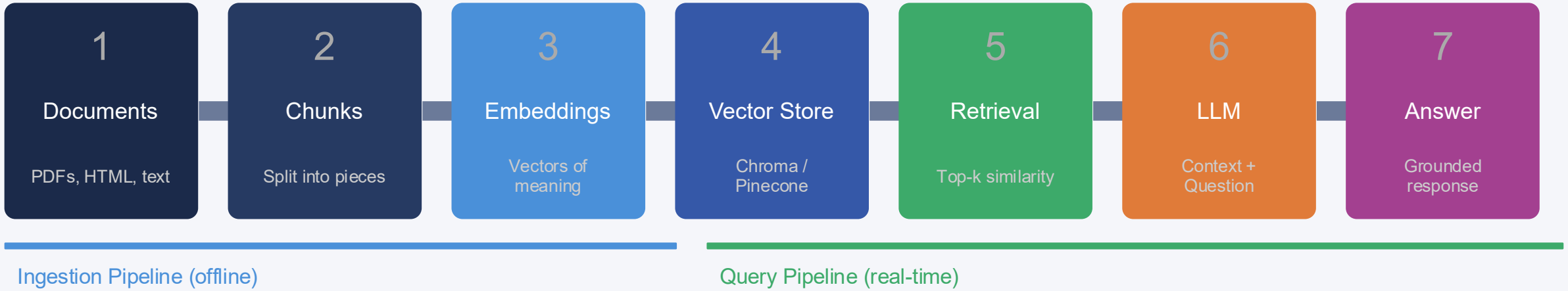
RAG vs Fine-Tuning vs Prompt Engineering

Choosing the right approach

Dimension	Prompt Engineering	Fine-Tuning	RAG
Use private/live data	No	No (static)	Yes
Training cost	None	High	Low
Latency	Lowest	Low	Medium
Knowledge updates	Manual	Re-train	Update vector store
Hallucination control	Limited	Limited	Strong (grounded)
Best for	Simple tasks	Domain style	Knowledge-intensive QA
Complexity	Very low	High	Medium

The RAG Pipeline

A 7-step end-to-end flow



Documents are processed offline once. At query time, the user's question is embedded, top-k chunks are retrieved, and the LLM generates a grounded answer.

Understanding Embeddings

How text becomes searchable vectors

Text → Numbers → Meaning

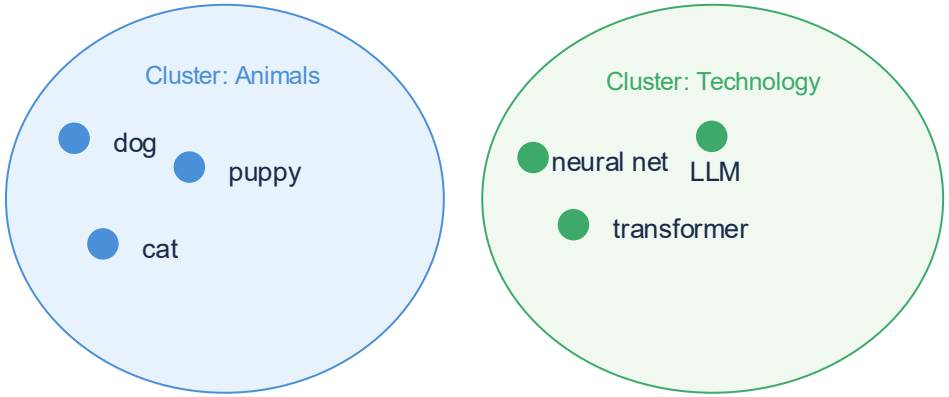
An embedding model converts any text into a fixed-length vector (e.g., 1536 numbers for OpenAI ada-002).

Texts with similar meanings produce vectors that are geometrically close in high-dimensional space.

This lets us find semantically relevant documents even when exact words differ.

Example: "dog" and "puppy" will be closer to each other than to "quantum mechanics".

Semantic Vector Space



Python – Embedding Example

```python
from openai import OpenAI
client = OpenAI()

response = client.embeddings.create(
    input="What is RAG?",
    model="text-embedding-ada-002"
)
vector = response.data[0].embedding
# → [0.0023, -0.0107, 0.0312, ...]  (1536 dims)
```

Vector Databases

Efficient similarity search at scale

What Vector DBs Do

Store	Index millions of high-dimensional vectors with metadata
Search	Find the k most similar vectors to a query vector in milliseconds
Scale	ANN (Approximate Nearest Neighbor) algorithms make this fast at scale
Filter	Metadata filtering to narrow search by category, date, or source

Cosine Similarity

$$\cos(\theta) = (A \cdot B) / (|A| \times |B|)$$

1.0 → Identical meaning
0.0 → Unrelated
-1.0 → Opposite meaning

Similarity Search Flow:

Query vector → Compare all vectors → Rank by score → Return top-k

Popular Options:

ChromaDB (local) | Pinecone (cloud) | Weaviate | Qdrant | FAISS

Chunking Strategies

How you split documents matters

Fixed-Size Chunking

`Split every N characters or tokens`

Pros

+ Simple to implement
+ Predictable chunk sizes
+ Works well for uniform text

Cons

- Can split mid-sentence
- No semantic awareness

Tip: Good default for PDFs

Recursive Character Split

`Split by \n\n → \n → space → char`

Pros

+ Respects natural boundaries
+ Paragraph-aware
+ LangChain default

Cons

- Still no semantic context
- Slightly more complex

Tip: Recommended starting point

Semantic Chunking

`Split at embedding-space boundaries`

Pros

+ Chunks carry full ideas
+ Better retrieval quality
+ Robust to varied formats

Cons

- Slower (needs embeddings)
- Higher cost

Tip: Use for high-value corpora

Module 2

Hands-On:

Build Your First RAG System

LangChain + OpenAI + ChromaDB • End-to-End Pipeline

Setup | Load Docs | Chunk | Embed + Store | Build Chain | Demo

Setup

Install dependencies and configure API access

Install Packages

```
Terminal
pip install langchain langchain-openai
pip install langchain-community
pip install chromadb
pip install pypdf
pip install openai python-dotenv
```

Python Imports

```
Python
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
import os
```

Configure API Key

```
Environment
# .env file
OPENAI_API_KEY=sk-...

# In Python
from dotenv import load_dotenv
load_dotenv()

# Or directly:
import os
os.environ['OPENAI_API_KEY'] = 'sk-...'
```

Key Components

LangChain	The orchestration framework - glues together loaders, splitters, embeddings, vector stores, and LLMs.
OpenAI	Provides both the embedding model (ada-002) and the completion model (gpt-5).
ChromaDB	Local vector database — zero setup, runs in-process. Perfect for prototyping.

Step 1: Load Documents

Ingesting source material with LangChain loaders

STEP 1 / 4

Python – Document Loading

```python
from langchain_community.document_loaders import (
    PyPDFLoader,
    DirectoryLoader,
    WebBaseLoader
)

# Load a single PDF
loader = PyPDFLoader("company_docs.pdf")
docs = loader.load()

# Or load a whole directory of PDFs
loader = DirectoryLoader(
    "./docs",
    glob="**/*.pdf",
    loader_cls=PyPDFLoader
)
docs = loader.load()
print(f"Loaded {len(docs)} documents")
```

What document loaders do:

`PyPDFLoader`

Extracts text from PDF files page by page

`WebBaseLoader`

Fetches and parses content from web URLs

`CSVLoader`

Reads tabular data from CSV files

`JSONLoader`

Parses structured JSON with jq-style paths

`DirectoryLoader`

Recursively loads all files in a folder

LangChain supports 100+ loaders: Notion, Google Drive, Confluence, SharePoint, and more.

Step 2: Chunk Documents

Splitting text into retrievable units

STEP 2 / 4

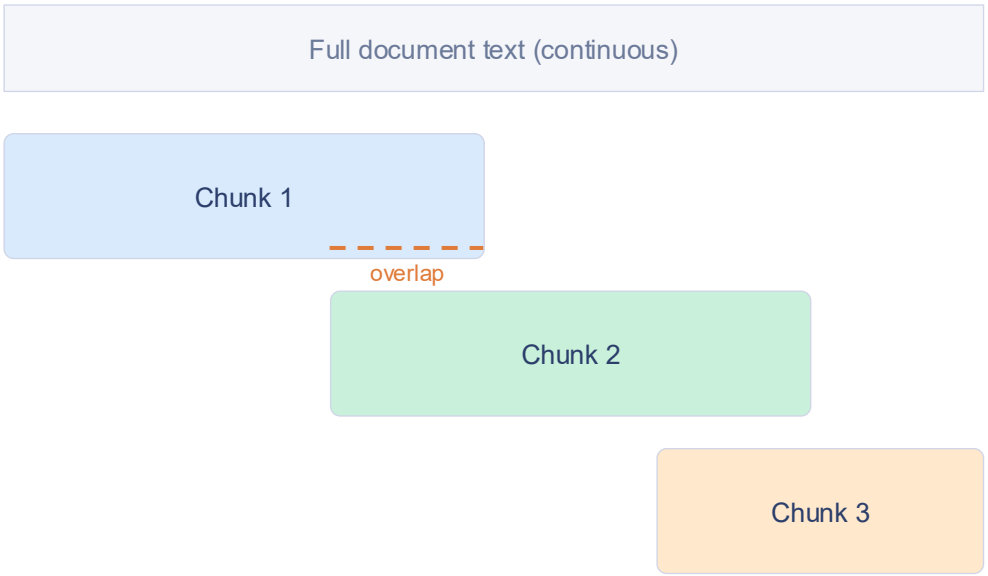
```python
Python – Text Splitting
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter
)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,    # characters per chunk
    chunk_overlap=200,  # overlap between chunks
    separators=["\n\n", "\n", " ", ""]
)

chunks = splitter.split_documents(docs)
print(f"Created {len(chunks)} chunks")
print(f"Sample chunk:\n{chunks[0].page_content}")
```

Key Parameters

chunk_size = ~1000 chars — Each chunk is ~1000 characters

chunk_overlap = ~200 chars — Overlap preserves context at boundaries

Chunk Overlap Visualization



- Larger chunks = more context, harder for LLM to focus
- Smaller chunks = more precise retrieval, risk losing context
- Overlap ensures sentences at chunk boundaries aren't lost
- Start with chunk_size=1000, overlap=200 — tune from there

Step 3: Create Embeddings & Store

Turning chunks into searchable vectors

STEP 3 / 4

Python – Embeddings + ChromaDB

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# Initialize the embedding model
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

# Create vector store from chunks
# This embeds all chunks and stores them
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"  # save to disk
)
print("Vector store created successfully!")
```

What happens under the hood:

1. **Each chunk sent to OpenAI API**
   API call per chunk (batched automatically)
2. **Embedding model returns vector**
   1536-dimensional float array
3. **Vector + metadata stored**
   Text, source, page number, etc.
4. **Index built for fast search**
   ANN index for millisecond queries

Step 4: Build the RAG Chain

Connecting retrieval and generation

STEP 4 / 4

Python – RAG Chain

```python
from langchain_openai import ChatOpenAI
from langchain.chains import RetrievalQA

# 1. Create the retriever
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 4}  # top 4 chunks
)

# 2. Create the LLM
llm = ChatOpenAI(
    model="gpt-5",
    temperature=0  # deterministic answers
)

# 3. Build the chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
```

How the chain works:

User Question	" What is the multi-head attention mechanism?"
Retriever	Embeds question → finds top-4 chunks from vector store
Prompt Builder	Assembles: [context chunks] + [question] → prompt
LLM	gpt-5 reads context and generates grounded answer
Answer	Factual response + source document references

Demo: Ask Questions

Your first RAG system in action

```python
Python – Querying

# Ask a question
result = qa_chain.invoke(
    {"query": "How will the OHS Regulations affect my business?"}
)

print(result["result"])
A: The OHS Regulations will impact your business on various levels. With
fewer approvals required from the Chief Safety Officer…

print("\nSources:")
for doc in result["source_documents"]:
    print(f"  - {doc.metadata['source']}, page {doc.metadata['page']}")
# → FAQ.pdf, page 0
```

Example Interactions:

Q: Who is going to approve it?

A: 8

Q: How much will it cost me to comply with the OHS Regulations?

A: It depends. Complying with the OHS Regulations is a progressive step in your safety program…

What just happened

1. Question was embedded into a 1536-dim vector
2. ChromaDB found the 4 most similar chunks
3. Chunks inserted into prompt as context
4. GPT-5 read the context and answered factually
5. Source documents returned for citation

Congratulations — you just built a working RAG system!

With RAG vs Without RAG

Grounded answers vs hallucination

Without RAG

With RAG

Question: "Who is going to approve the complaint?"

LLM Response:

" Do you mean a complaint or a compliance request? Approval depends on context: - Workplace/HR complaint: HR or the ethics/compliance officer…”

HALLUCINATION

- ✗ Based on generic internet training data
- ✗ No reference to your actual contract
- ✗ Evidence are completely fabricated
- ✗ Could expose you to legal liability

LLM Response:

" The Chief Safety Officer (though the new Regulations require fewer approvals from the CSO).…"

GROUNDED ANSWER

- ✓ Retrieved from FAQ.pdf, p.0
- ✓ Exact information from the document
- ✓ Source citation provided for verification
- ✓ Safe to use in business decisions

10-Minute Break

We'll resume at Module 3: Advanced RAG Techniques

Module 3

Level Up:

Advanced RAG Techniques

Re-Ranking • Multi-Query • Hybrid Search • Memory • Evaluation

Re-Ranking

The highest ROI improvement in most RAG systems

Retrieve broadly. Rank precisely. Send only the best context to the LLM.

20 chunks

Initial Retrieval

`k=20 chunks`

Fast ANN search
Recall > Precision

Rank

Re-Ranker

`Cross-encoder model`

Scores each chunk
against the query

4 chunks

LLM Context

`Top k=4 chunks`

High precision
Lower token cost

Python – Re-Ranking

```python
from langchain.retrievers import \
    ContextualCompressionRetriever
from langchain_cohere import CohereRerank

compressor = CohereRerank(top_n=4)
retriever = ContextualCompressionRetriever(
  base_compressor=compressor,
  base_retriever=vs.as_retriever(k=20))
```

Why it works:

Embedding search optimizes for recall (cast a wide net). Re-rankers use cross-encoders to score query+chunk jointly, optimizing for precision. The combination dramatically outperforms either alone.

Multi-Query Retrieval

Overcome the limits of a single query formulation

The Problem with Single-Query RAG:

Your embedding might miss relevant chunks that describe the same thing using different words. Multi-query generates several variants, retrieves for each, and fuses results.

"'What is the easiest way to find out how to be compliant?"

- "What is the simplest way to identify and meet the compliance requirements that apply to me?"
- "Where can I find an easy, step-by-step checklist or guide to achieve compliance with relevant laws, standards, and policies?"
- "How can I quickly determine the actions needed to become and remain compliant in my organization?"

Fused, de-duplicated result set

Python - Multi-Query

```python
from langchain.retrievers.multi_query import
MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-5")

multiquery_retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(k=3),
    llm=llm
)
# Automatically generates 3 query variants,
# retrieves for each, and fuses results
results = multiquery_retriever.invoke(
    "'What is the easiest way to find out how to be
compliant?"
)
```

When to use:

Complex questions where a single embedding may miss relevant chunks due to phrasing differences. Adds ~2-3x LLM calls but significantly improves recall.

Hybrid Search

BM25 keyword + semantic search combined

BM25 (Keyword)

TF-IDF based sparse retrieval

Strengths:

+ Exact term matching

+ Great for IDs, names, codes

+ Fast, no embeddings

Weaknesses:

- Misses synonyms

- No semantic understanding

Semantic Search

Dense vector similarity

Strengths:

+ Understands intent

+ Handles synonyms

+ Concept-level matching

Weaknesses:

- Misses exact terms

- Embedding cost

Reciprocal Rank Fusion (RRF): Merge rankings from both retrieval methods using a formula that rewards results appearing in both ranked lists. Typically outperforms either method alone by 15-30%.

Conversational RAG

Adding memory and chat history

The Challenge:

Follow-up questions like "Who is going to approve it?" or "How much will it cost me to comply with the OHS Regulations?" lose all context in a stateless RAG chain. We need to track conversation history.

User: Who is going to approve it?

AI: The Chief Safety Officer will approve fewer items under the new OHS Regulations…

User: How much will it cost me to comply with the OHS Regulations?

AI: It depends. Complying with the OHS Regulations is a progressive step in your safety program…

Python – Conversational RAG

```python
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
    output_key="answer"
)

conv_chain = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=retriever,
    memory=memory,
    return_source_documents=True
)

# First question
r1 = conv_chain.invoke({"question": "How will the OHS
Regulations affect my business?"})

# Follow-up — memory handled automatically
r2 = conv_chain.invoke({"question": " Who is going to
approve it?"})
```

Memory types: ConversationBufferMemory (full history) • ConversationSummaryMemory (compressed) • WindowMemory (last N turns)

RAG Evaluation

Measuring quality with RAGAS

RAGAS — Retrieval Augmented Generation Assessment: framework for automated evaluation of RAG pipelines without human labels

Faithfulness

Score: 0.0 – 1.0

Are all claims in the answer supported by the retrieved context? Detects hallucinations introduced after retrieval.

High score = LLM not making things up

Answer Relevancy

Score: 0.0 – 1.0

Is the answer relevant to the question? Checks if the answer stays on topic and is complete.

High score = Answer matches what was asked

Context Precision

Score: 0.0 – 1.0

Were the retrieved chunks relevant? Measures the signal-to-noise ratio of retrieved context.

High score = Retrieval is precise, not noisy

Python – RAGAS Evaluation

```python
from ragas import evaluate
from ragas.metrics.collections import ContextPrecision, Faithfulness, AnswerRelevancy

results = evaluate(
  dataset=eval_data,
  metrics=[faithfulness])
print(results)  # pip install ragas
```

Module 4

RAG in Production

& The Future

Agentic RAG • Graph RAG • Multimodal • Production Stack

The RAG Evolution

From naive to agentic

Generation 1

Naive RAG

Simple retrieve-and-generate. Single query, fixed-size chunks, no reranking.

- PyPDF + RecursiveTextSplitter
- OpenAI embeddings
- ChromaDB / FAISS
- RetrievalQA chain
- No evaluation

What you built today

Generation 2

Advanced RAG

Improved retrieval quality through multi-query, re-ranking, and hybrid search.

- Semantic + BM25 hybrid
- Cross-encoder re-ranking
- Multi-query expansion
- Conversational memory
- RAGAS evaluation

Production-ready

Generation 3

Agentic RAG

LLM as orchestrator — decides when and what to retrieve, uses tools, self-corrects.

- LLM plans retrieval steps
- Multiple tool use
- Self-reflection & critique
- Graph + vector fusion
- Autonomous quality check

Frontier systems

Agentic RAG

The LLM as an intelligent orchestrator

What changes with Agentic RAG:

Plan	LLM decides WHAT to retrieve and in what order — not a fixed pipeline
Act	Multiple tool calls: vector search, SQL, web search, calculators
Observe	LLM inspects retrieval results and decides if more context is needed
Reflect	LLM self-critiques the draft answer and re-retrieves if unsatisfied

LangGraph and LlamaIndex provide frameworks for building agentic RAG systems with reliable orchestration loops.

Python – Agentic RAG (LangGraph)

```python
from langgraph.graph import StateGraph
from langchain.tools.retriever import create_retriever_tool

# Define retrieval tool
retriever_tool = create_retriever_tool(
    retriever,
    "search_documents",
    "Search company documents for information"
)

# The LLM decides when and what to retrieve
# using ReAct-style planning
agent = create_react_agent(
    llm=ChatOpenAI(model="gpt-5"),
    tools=[retriever_tool]
)

result = agent.invoke(
    {"messages": [HumanMessage(content=query)]}
)
```

Graph RAG & Multimodal RAG

Beyond text: structured knowledge and rich media

Graph RAG

Microsoft GraphRAG — knowledge graph-enhanced retrieval

- Extract entities and relationships from documents
- Build a knowledge graph (nodes = entities, edges = relationships)
- Retrieve by graph traversal, not just vector similarity
- Excel at multi-hop questions: "Who reported to the CEO who approved X?"
- Handles structured data (org charts, taxonomies) far better than flat RAG

```
pip install graphrag
```

Multimodal RAG

Retrieve and reason over images, tables, charts, and text

- Process PDFs with figures, diagrams, and complex tables
- Vision LLMs (GPT-4o, Gemini) understand images as context
- Use Unstructured.io to extract tables/images from PDFs
- Embed images using CLIP or OpenAI vision embeddings
- Answer: "What does the architecture diagram on page 4 show?"

```
pip install unstructured
```

The Modern RAG Stack

Full production architecture

Data Ingestion

- Document loaders (LangChain)
- Unstructured.io (PDFs, tables)
- Scheduled pipelines (Airflow)

Chunking & Indexing

- Recursive / semantic chunking
- Metadata enrichment
- Multi-vector indexing

Hybrid Retrieval

- Dense (OpenAI ada-002)
- Sparse (BM25 / Elasticsearch)
- RRF fusion

Re-Ranking

- Cohere Rerank / FlashRank
- Cross-encoder scoring
- Top-k selection

Generation

- GPT-4o / Claude 3.5
- Structured prompting
- Citation injection

Guardrails & Eval

- RAGAS metrics (CI/CD)
- LlamaGuard / Guardrails AI
- Hallucination detection

Key Takeaways

What you learned today

01 RAG solves a real problem

LLMs don't know your private data. RAG grounds them in your documents, eliminating hallucinations for knowledge-intensive tasks.

02 The pipeline is simpler than you think

Load → Chunk → Embed → Store → Retrieve → Generate. LangChain + OpenAI + ChromaDB makes this < 30 lines of code.

03 Quality lives in retrieval, not generation

Most RAG failures are retrieval failures. Re-ranking, multi-query, and hybrid search are your highest-ROI improvements.

04 Evaluate before you ship

RAGAS gives you automated metrics — faithfulness, relevancy, precision — that you can run in CI/CD to catch regressions.

05 The frontier is agentic and multimodal

RAG is evolving from a fixed pipeline to an intelligent agent that plans, reflects, and uses diverse tools to answer any question.

Resources

Keep learning after the workshop

Official Documentation

LangChain Docs
python.langchain.com/docs/get_started/introduction

OpenAI API Reference
platform.openai.com/docs

ChromaDB Docs
docs.trychroma.com

RAGAS Documentation
docs.ragas.io/en/stable

LlamaIndex Docs
docs.llamaindex.ai

Papers & Reading

RAG: Lewis et al. (2020)
arxiv.org/abs/2005.11401

GraphRAG: Microsoft (2024)
arxiv.org/abs/2404.16130

RAGAS Evaluation Paper
arxiv.org/abs/2309.15217

Advanced RAG Survey
arxiv.org/abs/2312.10997

Agentic RAG Overview
arxiv.org/abs/2501.09136

Video & Courses

DeepLearning.AI: LangChain
www.deeplearning.ai/short-courses/langchain-for-llm-application-development

DeepLearning.AI: RAG Course
www.deeplearning.ai/short-courses/building-and-evaluating-advanced-rag

LangChain YouTube Channel
www.youtube.com/@LangChain

Andrej Karpathy: Intro to LLMs
www.youtube.com/watch?v=zjkBMFhNj_g

Sam Witteveen RAG Tutorials
www.youtube.com/@samwitteveenai

Thank You

Questions & Answers